



Universidad Autónoma de Yucatán

Facultad de Matemáticas

Sistemas Distribuidos

ADA 07

- Ashley Shaden Aguilar Moreno
- Monserrat Anaya Alva
- Mariano Marcel Calderón Núñez

Profesor: Enrique Ayala Franco

Fecha: 9 de mayo de 2026

Web Service con REST

Simulación de exclusión mutua distribuida con servidor central

Etap 1. Verificar el funcionamiento del programa con las operaciones actuales, usar Postman o Bruno, para probar los siguientes escenarios:

Caso 1: Concurrencia

Saldo inicial en cuenta 1: 1000

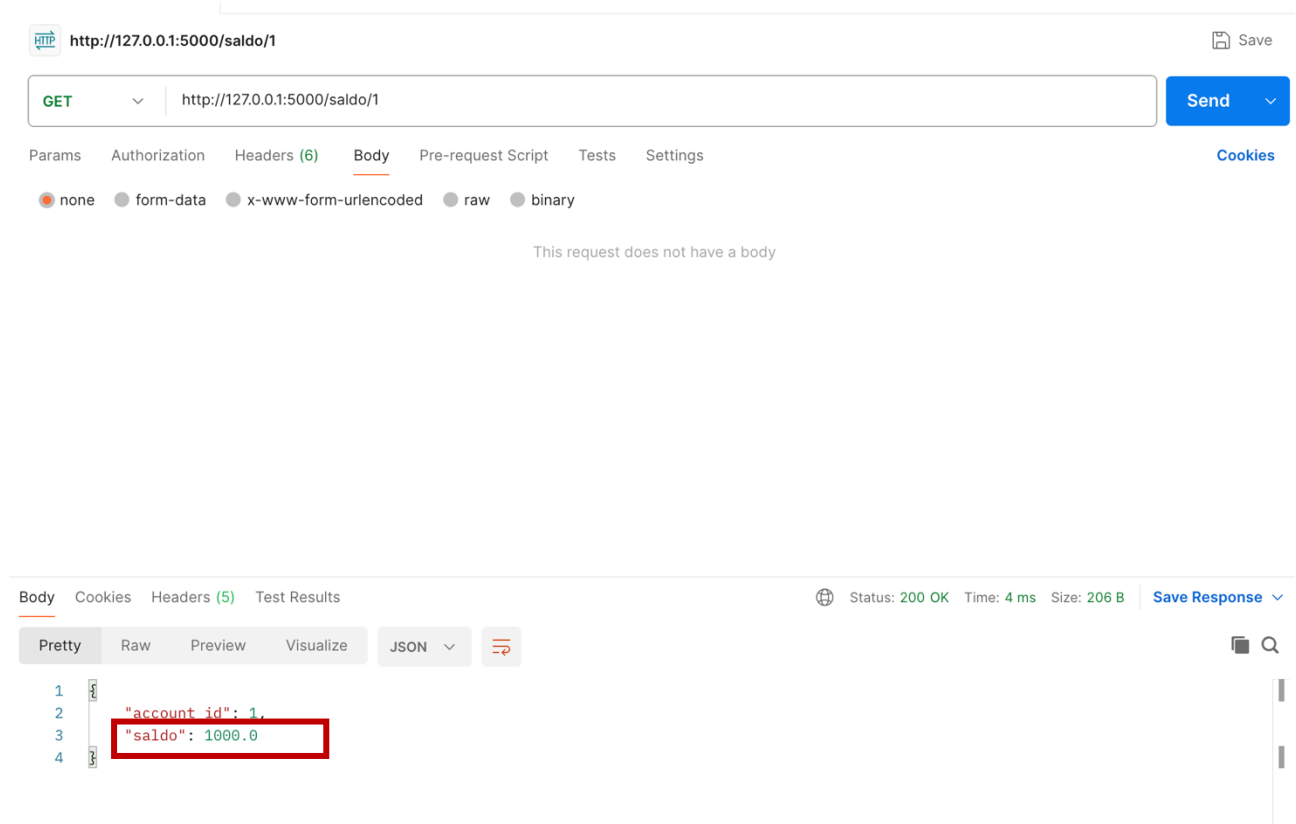
Clientes simultáneos:

- Cliente A: retirar 500
- Cliente B: retirar 700

Resultado:

- A → saldo = 500
- B → error
- Usar el endpoint *resultado* y *saldo*.

Primero debemos verificar el saldo de la cuenta el cual es \$1000.



Posteriormente, en dos ventanas distintas de Postman, se realizan dos solicitudes POST para retirar dinero: la primera por \$500 y la segunda por \$700. Ambas peticiones se envían de manera casi simultánea con el fin de simular concurrencia. Como respuesta, el sistema

devuelve un request_id y un estado inicial de pending, los cuales se utilizan posteriormente para consultar y validar el resultado de cada operación.

The image displays two screenshots of a REST client interface, likely Postman, showing the process of sending a POST request to a withdrawal endpoint and receiving a response.

Top Screenshot:

- Request:** A POST request to `http://127.0.0.1:5000/retirar`. The body is a JSON object: `{ "account_id": 1, "monto": 700 }`.
- Response:** The response is a JSON object: `{ "request_id": "7606ffd9-8c77-4896-9dc7-d836668a2bab", "status": "pending" }`. The response fields are highlighted with a red box.

Bottom Screenshot:

- Request:** A POST request to `http://127.0.0.1:5000/retirar`. The body is a JSON object: `{ "account_id": 1, "monto": 500 }`.
- Response:** The response is a JSON object: `{ "request_id": "669f4eb4-ca5f-412b-bd40-1b49a3f73ac3", "status": "pending" }`. The response fields are highlighted with a red box.

Con los request_id obtenidos, se procede a validar si las operaciones fueron procesadas correctamente. El primer retiro de \$500 debe presentar un estado done, mientras que el segundo retiro de \$700 debe devolver un error, debido a que el saldo inicial de la cuenta es de \$1000, lo que impide cubrir ambas operaciones.

The screenshot displays a REST client interface. At the top, the URL bar shows `http://127.0.0.1:5000/resultado/669f4eb4-ca5f-412b-bd40-1b49a3f73ac3`. Below the URL bar, the request method is set to `GET`. The `Body` tab is selected, showing the message "This request does not have a body".

Below the request configuration, the response is displayed. The `Body` tab is selected, showing the response in `JSON` format. The response status is `200 OK`, with a time of `6 ms` and a size of `206 B`. The response body is a JSON object:

```
1 {  
2   "saldo": 500.0,  
3   "status": "done"  
4 }
```

The JSON object is highlighted with a red box, indicating the successful completion of the operation.

GET http://127.0.0.1:5000/resu POST http://127.0.0.1:5000/reli POST http://127.0.0.1:5000/reli + ...

HTTP http://127.0.0.1:5000/resultado/7606ffd9-8c77-4896-9dc7-d836668a2bab Save

GET http://127.0.0.1:5000/resultado/7606ffd9-8c77-4896-9dc7-d836668a2bab Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (5) Test Results Status: 200 OK Time: 4 ms Size: 226 B Save Response

Pretty Raw Preview Visualize JSON

```
1
2 "mensaje": "Fondos insuficientes",
3 "status": "error"
4
```

Por ultimo, verificamos el saldo final de la cuenta, el cual debería ser \$500.

GET http://127.0.0.1:5000/saldi POST http://127.0.0.1:5000/reli POST http://127.0.0.1:5000/reli + ...

HTTP http://127.0.0.1:5000/saldo/1 Save

GET http://127.0.0.1:5000/saldo/1 Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (5) Test Results Status: 200 OK Time: 6 ms Size: 205 B Save Response

Pretty Raw Preview Visualize JSON

```
1
2 "account_id": 1,
3 "saldo": 500.0
4
```

Caso 2: Depósito inválido

- monto = -100

Resultado:

- error

A continuación, se procede a realizar un depósito inválido. Para ello, se realiza una llamada al endpoint correspondiente y, en el cuerpo de la solicitud (formato JSON), se envían los datos necesarios, incluyendo un monto negativo con el fin de validar que el sistema rechace la operación.

The screenshot displays a REST client interface for a POST request to the endpoint `http://127.0.0.1:5000/depositar`. The request body is a JSON object with `"account_id": 1` and `"monto": -100`, where the negative amount is highlighted with a red box. The response status is `200 OK` with a time of `7 ms` and a size of `247 B`. The response body is a JSON object with `"request_id": "d91b853e-0919-408d-9faa-5a273d146aaa"` and `"status": "pending"`.

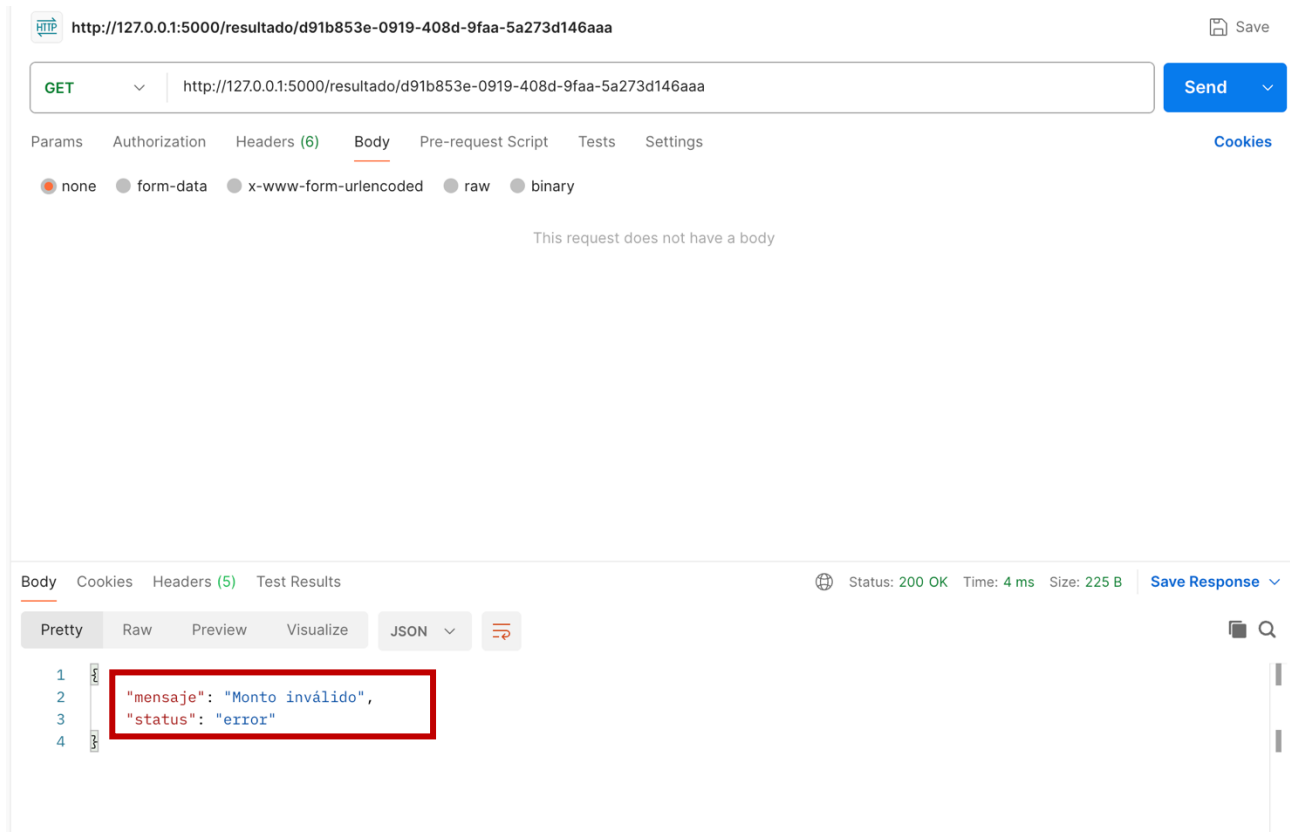
```
POST http://127.0.0.1:5000/depositar
```

```
{  "account_id": 1,  "monto": -100}
```

Status: 200 OK Time: 7 ms Size: 247 B

```
{  "request_id": "d91b853e-0919-408d-9faa-5a273d146aaa",  "status": "pending"}
```

Posteriormente, se consulta el endpoint de resultados, donde se verifica que el depósito no pudo ser procesado debido a que el monto ingresado es inválido.



Caso 3: Cuenta inactiva

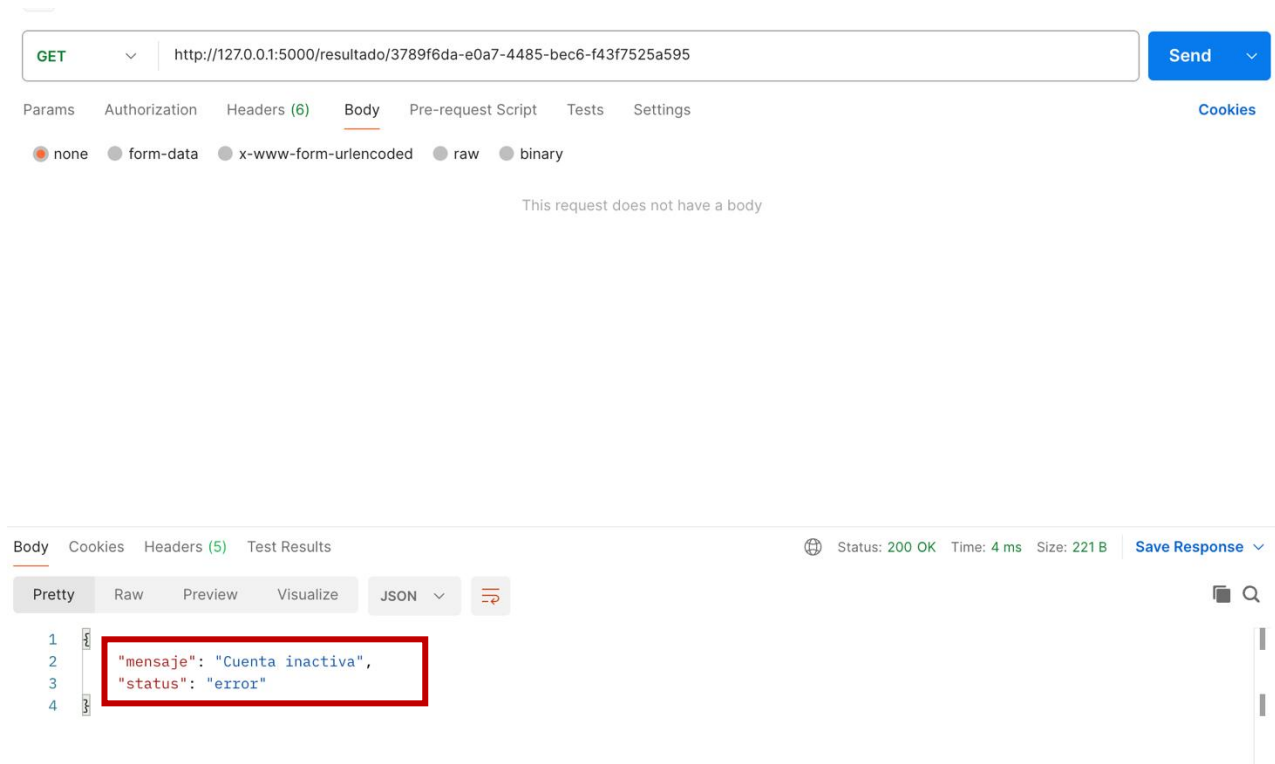
Tratar de retirar en cuenta 2

- estado = "bloqueada"
- intento de retiro

Resultado:

- error

Finalmente, se intenta realizar un retiro en una cuenta con estado bloqueado. Al ejecutar la operación y consultar su resultado mediante el endpoint correspondiente, se observa que el sistema devuelve un error, debido a que la cuenta se encuentra inactiva.



Etapas 2: Implementar en la API REST:

- Al menos un endpoints más con operaciones que puedan ser útiles para los usuarios o administradores (lista de cuentas, alta de nueva cuenta, activar/desactivar cuenta, etc.)
- Aplicar validaciones para datos proporcionados.
- Implementar el que los endpoints solo puedan ser accedidos si se cuenta con un token, incluir el endpoint login (usar jwt).

Evidencia: describir endpoints y los cambios realizados, mostrar capturas de pantalla de las pruebas realizadas.

Implementación de Seguridad (JWT): Se implementó el estándar JSON Web Token (JWT) utilizando la librería PyJWT.

- POST /login: Recibe las credenciales de un usuario y devuelve un token con un tiempo de expiración de 1 hora.
- Se creó el decorador `@token_required` que intercepta las peticiones, busca el token en el encabezado `Authorization: Bearer <token>` y valida que no esté expirado ni alterado antes de permitir el acceso a los demás endpoints.

Nuevos Endpoints Implementados

- GET /cuentas: Retorna una lista con la información de todas las cuentas registradas en el sistema (ID, nombre y saldo).
- POST /crear_cuenta: Permite dar de alta una nueva cuenta en el sistema de almacenamiento.

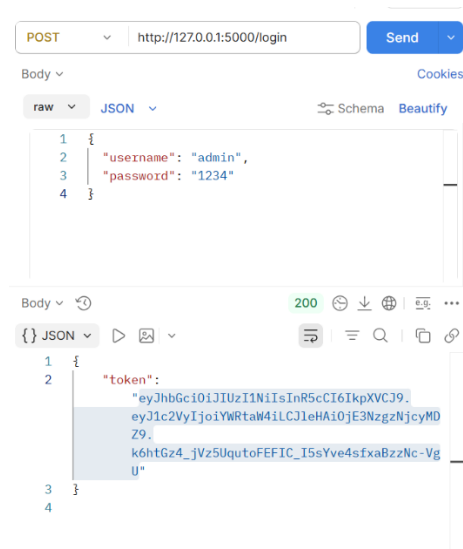
3. POST /transferir: Permite enviar dinero de una cuenta a otra, garantizando que los saldos se actualicen de forma coherente.

Validaciones Aplicadas

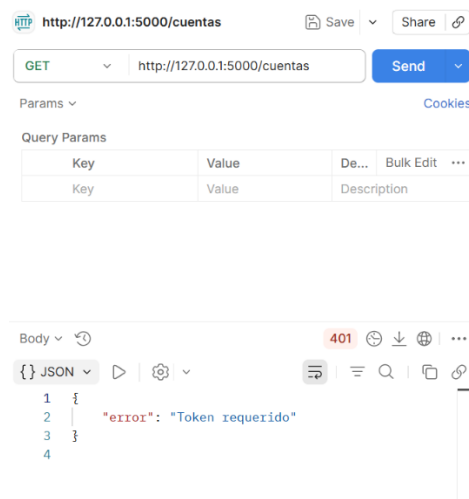
- En todos los endpoints POST se verifica que el cuerpo de la petición contenga un JSON válido.
- Se validó la existencia de los campos obligatorios (ej. nombre, saldo, from_account_id).
- En /crear_cuenta se valida que el nombre no esté vacío, que el saldo sea de tipo numérico y que no sea negativo.
- Se implementó la función validar_monto para asegurar que los depósitos, retiros y transferencias sean estrictamente números mayores a cero.
- En /transferir se valida que la cuenta origen tenga fondos suficientes y que no se permita transferir a la misma cuenta.

Evidencias:

Generación de Tokens



Protección de Rutas



Lista de Cuentas Exitosas

HTTP

http://127.0.0.1:5000/cuentas

Save

Share

GET

http://127.0.0.1:5000/cuentas

Send

Authorization

Auth Type

Bearer To...

Token

.....

The authorization header will be automatically generated when you send the request. Learn

Body

200

JSON

```
1 [
2   {
3     "account_id": 1,
4     "nombre": "Juan Perez",
5     "saldo": 500.0
6   },
7   {
8     "account_id": 2,
9     "nombre": "Maria Lopez",
10    "saldo": 500.0
11  }
12 ]
```

Verificación y Creación de cuentas

HTTP

http://127.0.0.1:5000/crear_cuenta

Save

Share

POST

http://127.0.0.1:5000/crear_cuenta

Send

Body

raw

JSON

Schema

Beautify

```
1 {
2   "nombre": "Carlos Slim",
3   "saldo": 5000
4 }
```

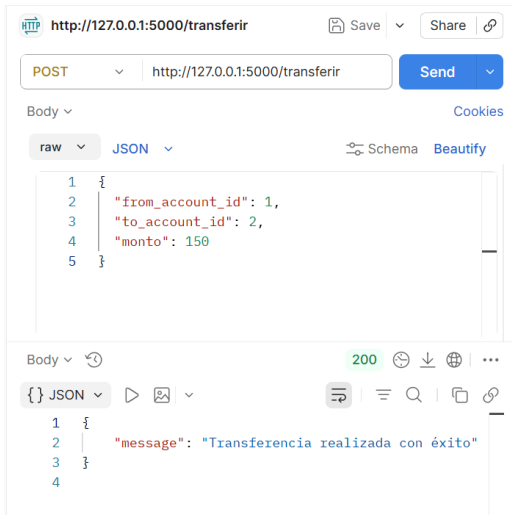
Body

201

JSON

```
1 {
2   "account_id": 3,
3   "nombre": "Carlos Slim",
4   "saldo": 5000
5 }
6
```

Transferencias



Etapas 3: Implementar un cliente que permita probar de manera interactiva las operaciones del sistema.

- Puede ser con una interfaz CLI, GUI o Web.
- Mostrar menú con las opciones, incluyendo la opción autenticar.
- Probar con al menos tres clientes simultáneos, accediendo a la misma cuenta.

Evidencia: mostrar capturas de pantalla de uso de las operaciones disponibles en el sistema y resultados en los clientes. Mostrar salidas en el servidor.

Inicio de Sesión

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 1

--- INICIO DE SESIÓN ---
Usuario (ej. admin): admin
Contraseña (ej. 1234): 1234
Login exitoso. Token guardado para esta sesión.
```

Alta de nueva cuenta

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 6

--- ALTA DE NUEVA CUENTA ---
Nombre del titular: Mariano Calderon
Saldo inicial: 2000

Respuesta del servidor: {'account_id': 4, 'nombre': 'Mariano Calderon', 'saldo': 2000.0}
```

Lista de Cuentas

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 8

--- LISTA DE CUENTAS ---

Respuesta del servidor: [{'account_id': 1, 'estado': 'activa', 'nombre': 'Juan Perez', 'saldo': 350.0}, {'account_id': 2, 'estado': 'bloqueada', 'nombre': 'Maria Lopez', 'saldo': 650.0}, {'account_id': 3, 'estado': 'activa', 'nombre': 'Carlos Slim', 'saldo': 5000}, {'account_id': 4, 'estado': 'activa', 'nombre': 'Mariano Calderon', 'saldo': 2000.0}]
```

Consultar Saldo

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 2
ID de la cuenta: 4

Respuesta del servidor: {'account_id': 4, 'saldo': 2000.0}
```

Depositar

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 3
ID de la cuenta: 4
Monto a depositar: 200

Respuesta del servidor: {'request_id': '7002280f-cb00-489d-bd8a-7ba590f6b154', 'status': 'pending'}
```

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 2
ID de la cuenta: 4

Respuesta del servidor: {'account_id': 4, 'saldo': 2200.0}
```

Retirar

```
PS C:\Users\calde\Downloads\REST_WS-main\REST_WS-main> python cliente_interactivo.py
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 4
ID de la cuenta: 4
Monto a retirar: 1000

Respuesta del servidor: {'request_id': '5827cfef-d794-47cc-93ea-a6d1d61af5b2', 'status': 'pending'}
```

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 2
ID de la cuenta: 4

Respuesta del servidor: {'account_id': 4, 'saldo': 1200.0}
```

Consultar Resultado

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 5
ID de la petición (request_id): 5827cfef-d794-47cc-93ea-a6d1d61af5b2

Respuesta del servidor: {'saldo': 1200.0, 'status': 'done'}
```

Cambiar Estado de Cuenta

```
=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 7

--- CAMBIAR ESTADO DE CUENTA ---
ID de la cuenta: 3
Estados disponibles: 1. Activa, 2. Bloqueada
Selecciona el nuevo estado (1 o 2): 2

Respuesta del servidor: {'message': 'Estado de la cuenta 3 actualizado a bloqueada'}
```

```

=====
SISTEMA DISTRIBUIDO - CAJA DE AHORRO
=====
1. Autenticar (Login)
2. Consultar Saldo
3. Depositar
4. Retirar
5. Consultar Resultado (Polling)
6. Crear Nueva Cuenta
7. Activar/Desactivar Cuenta
8. Listar Todas las Cuentas
9. Salir
Selecciona una opción: 8

--- LISTA DE CUENTAS ---

Respuesta del servidor: [{'account_id': 1, 'estado': 'activa', 'nombre': 'Juan Perez', 'saldo': 350.0}, {'account_id': 2, 'estado': 'bloqueada', 'nombre': 'Maria Lopez', 'saldo': 650.0}, {'account_id': 3, 'estado': 'bloqueada', 'nombre': 'Carlos Slim', 'saldo': 5000}, {'account_id': 4, 'estado': 'activa', 'nombre': 'Mariano Calderon', 'saldo': 1200.0}]

```

Respuestas del servidor

```

127.0.0.1 - - [09/May/2026 16:51:49] "POST /crear_cuenta HTTP/1.1" 201 -
127.0.0.1 - - [09/May/2026 16:51:54] "GET /cuentas HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:52:17] "GET /saldo/4 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:52:48] "POST /depositar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:52:57] "GET /saldo/4 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:53:09] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:53:14] "GET /saldo/4 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:53:33] "GET /resultado/5827cfef-d794-47cc-93ea-a6d1d61af5b2 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:53:56] "POST /cambiar_estado HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:54:00] "GET /cuentas HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:21] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:21] "GET /saldo/1 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:21] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:21] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:21] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:22] "GET /resultado/bbdcd263-295d-4759-8dbe-eb4adac0bf8b HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:22] "GET /resultado/ee2917b9-eee4-48ba-a319-310ae24101b3 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:22] "GET /resultado/ec2173a1-ccc5-49a8-8645-0003e90cb383 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:57:22] "GET /saldo/1 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:33] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:33] "GET /saldo/1 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:33] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:33] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:33] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:35] "GET /resultado/abb49031-4476-4934-86dd-6c8da2e0997e HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:35] "GET /resultado/0b987f8e-270e-4d86-8bd1-348ab83c5a0f HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:35] "GET /resultado/2259f53e-826d-401b-a31a-88726f064251 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:58:35] "GET /saldo/1 HTTP/1.1" 200 -

```

Evidencia Concurrencia

Objetivo de la prueba:

Demostrar la capacidad del sistema distribuido para manejar peticiones simultáneas sobre un mismo recurso compartido (el saldo de una cuenta), garantizando la consistencia de los datos y evitando condiciones de carrera (race conditions) mediante exclusión mutua implícita.

Metodología:

Se desarrolló un script de automatización en Python (`prueba_concurrencia.py`) utilizando la librería `threading`. El script simula tres clientes concurrentes (Cliente A, B y C) que intentan realizar un retiro de \$500.00 MXN sobre la misma cuenta (Cuenta 1), la cual contaba con un saldo inicial de \$800.00 MXN en el momento de la prueba. Los tres hilos de ejecución disparan las peticiones exactamente en el mismo milisegundo hacia el endpoint asíncrono `POST /retirar`. Posteriormente, los clientes realizan polling (consultas periódicas) para conocer el estado de su solicitud.

Comportamiento del Sistema y Resultados:

Recepción Asíncrona: La API REST recibió las tres peticiones simultáneas sin bloqueos. A cada solicitud se le asignó un `request_id` único y el estado inicial `pending`, encolando los tres mensajes de manera exitosa.

Serialización (Cola de Mensajes): La estructura de datos de la cola (`FIFO - First In, First Out`) organizó las peticiones de manera estrictamente secuencial, sirviendo como mecanismo de control de concurrencia.

Procesamiento del Worker (Exclusión Mutua):

El Worker consumió el primer mensaje en llegar a la cola, accedió al recurso, validó que el saldo (\$800.00) era suficiente, procesó el retiro y guardó el saldo final de \$300.00 (cambiando el estado de la petición a `done`).

Al procesar los dos mensajes restantes en la cola, el Worker evaluó el estado actualizado de la cuenta (\$300.00). Como el retiro solicitado (\$500.00) era mayor al saldo disponible, el sistema aplicó correctamente la regla de negocio y rechazó ambas transacciones, actualizando su estado a error por "Fondos insuficientes".

Conclusión:

Los resultados de salida en los clientes confirman que la arquitectura propuesta (API REST + Cola de Mensajes + Worker) resuelve eficazmente el problema de acceso concurrente. La cola garantiza que el worker acceda a la memoria en un modelo de hilo único, logrando exclusión mutua sin necesidad de bloqueos explícitos (`locks`) a nivel de base de datos, y manteniendo la integridad y consistencia del saldo.


```
--- INICIANDO PRUEBA DE CONCURRENCIA (3 CLIENTES) ---
Saldo inicial en cuenta 1: {'account_id': 1, 'saldo': 800.0}
-----
[Cliente A] Solicitando retiro de 500...
[Cliente B] Solicitando retiro de 500...
[Cliente C] Solicitando retiro de 500...
[Cliente B] Petición en cola, request_id: 56919686-5eb5-4603-8a70-bb6aec4994e2
[Cliente A] Petición en cola, request_id: d5b95aa6-01e3-499c-825c-777fc4068d62
[Cliente C] Petición en cola, request_id: d58c9892-cff3-4e7a-a0d8-d0d8adf02433
[Cliente B] RESULTADO FINAL: {'saldo': 300.0, 'status': 'done'}
[Cliente A] RESULTADO FINAL: {'mensaje': 'Fondos insuficientes', 'status': 'error'}
[Cliente C] RESULTADO FINAL: {'mensaje': 'Fondos insuficientes', 'status': 'error'}
-----
Prueba de concurrencia finalizada.
Saldo final en cuenta 1: {'account_id': 1, 'saldo': 300.0}
```

```
-----
127.0.0.1 - - [09/May/2026 16:59:52] "GET /saldo/1 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:52] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:52] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:52] "POST /retirar HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:53] "GET /resultado/56919686-5eb5-4603-8a70-bb6aec4994e2 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:53] "GET /resultado/d5b95aa6-01e3-499c-825c-777fc4068d62 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:53] "GET /resultado/d58c9892-cff3-4e7a-a0d8-d0d8adf02433 HTTP/1.1" 200 -
127.0.0.1 - - [09/May/2026 16:59:53] "GET /saldo/1 HTTP/1.1" 200 -
```